SCALE FOR PROJECT FLY-IN (/PROJECTS/F

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-c1eef1ea-dd4
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify with the person (or the group) evaluated the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's GiT repository.

- Double-check that the GiT repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, non-functioning program, a norm error, cheating etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists. If you want to edit a file, take the time to explicit the reasons with the evaluated student and make sure both of you are okay with this.

- You must also verify the absence of memory leaks. Any memory allocated on the heap must

be properly freed before the end of execution.
You are allowed to use any of the different tools available on the computer, such as leaks, valgrind, or e_fence. In case of memory leaks, tick the appropriate flag.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/223998/en.subject.pdf)

maps.tar.gz (https://cdn.intra.42.fr/document/document/55897/maps.tar.gz)

Preliminaries

Basics

The review is done in the presence of the graded learner(s). This is how everyone progresses: by interacting with others.

- The learner(s) must be present during the defense.
- If the learner is not present, the review cannot proceed.
- As soon as a core functionality is non-functional, the review stops. You can look at the code sections, but they will not be graded.

Yes No

Work submission

Only grade the work that is in the learner's or group's Git repository. Check that only the requested files are available in the git repository. If not, the evaluation stops here.

Yes No

README.md

README.md file

Does the repository contain a README.md file at its root, and does it include all of the following?

- A "Description" section explaining the project's purpose and providing a brief overview.
- An "Instructions" section with details about how to run the project.
- An "Algorithm explanation" section describing the pathfinding approach and design decisions
- Documentation of visual representation features and how they enhance user experience.
- Example input and expected output demonstrating the program's functionality.

Yes No

Project Structure and Requirements

OOP

The project is completely object-oriented with proper class design and follows Python OOP best practices. Check for proper class separation, encapsulation, and inheritance where appropriate.

Yes No

Type safe

The project is completely typesafe and passes mypy type checking without errors. Run 'mypy .' in the project directory to verify. If mypy is not installed, check that type hints are present throughout the codebase.

Yes No

Graph implementation

The graph implementation is custom-built without using forbidden libraries
(such as networkx, graphlib, etc.). The learner should be able to explain how it works.

Yes No

Parser Implementation

Input files

Test the parser with valid input files. The parser correctly handles the new input file format including

- Number of drones using "nb_drones: <number>" format
- Zone definitions with type prefixes (start_hub:, end_hub:, hub:)
- Connection definitions using "connection: <zone1>-<zone2>" format
- Optional metadata with defaults (zone=normal, max_drones=1)
- Connection metadata with max_link_capacity=1 default
- Comments starting with '#' are ignored

At least 4 out of 5 format requirements should work correctly.

Yes No

Error handling

Test error handling in the parser. The parser correctly handles errors:

- Test with malformed files (missing drone count, invalid format)
- Test with invalid zone types (should reject with clear error messages)
- Test with missing start_hub or end_hub definitions
- Test with invalid capacity values (non-positive integers)
- Test with duplicate zone names or connections

At least 4 out of 5 error cases should be handled correctly with clear error messages.

Yes No

Zone and Movements Mechanics

Zone occupancy rules

Test zone occupancy rules with capacity constraints:

- Zones respect max_drones capacity (default 1)
- Drones cannot enter zones that would exceed capacity
- Multiple drones can share start and end zones
- Capacity constraints are properly enforced during simulation

Create test cases to verify these rules are enforced.

Yes No

Movement costs

Test movement costs based on zone types:

- normal zones: 1 turn (default)
- restricted zones: 2 turns
- priority zones: 1 turn (but preferred in pathfinding)
- blocked zones: inaccessible

Verify the simulation respects these movement costs in the output.

Yes No

Connection capacity

Test connection capacity constraints:

- Connections respect max_link_capacity defined in connection metadata (default 1)
- Multiple drones can traverse high-capacity connections simultaneously
- Connection capacity limits are enforced during movement

Verify capacity constraints work correctly in simulation.

Yes No

Visual Representation

Visualization features

Test visual representation features:

- The program provides a clear visual feedback (colored terminal output and/or graphical interf
- Colors specified in zone metadata are used for visualization
- Visual representation enhances understanding of the simulation
- The visual system clearly shows drone positions and movements

The implementation should demonstrate meaningful visual feedback.

Yes No

Basic Functionality Tests

Simple scenarios

Test with simple scenarios:
- Test with single drone on a linear path
- Test with multiple drones using different paths
- Test with provided example maps from attachments/
- Verify output format: each line represents one turn, format "D<ID>-<zone>" or "D<ID>-<connectio
- Verify stationary drones are omitted from output

At least 4 out of 5 basic tests should pass successfully.

Yes No

Simulation ends correctly

Test that the simulation ends correctly when all drones reach the end zone.
The program should stop producing output once all drones have arrived.

Yes No

Pathfinding Algorithm

Valid path

The pathfinding algorithm successfully finds valid paths from
start to end zone for various test cases. Test with:

- Simple linear maps
- Maps with multiple possible paths
- Maps with bottlenecks and capacity constraints
- Maps using different zone types

The algorithm should find valid solutions for most test cases.

Yes No

Conflict resolution

Test conflict resolution and capacity management:

- Test scenarios where drones compete for limited capacity
- Verify the algorithm handles multiple drones simultaneously
- Check that capacity constraints prevent conflicts
- Test turn-based movement with multi-turn zones (restricted)

The algorithm should demonstrate proper capacity-aware conflict resolution.

Yes No

Performance and Optimization

Efficiency

Test with more complex scenarios to evaluate performance:

- Test with higher drone counts (10+ drones)
- Test with complex map topologies and capacity constraints
- Verify the solution minimizes total simulation turns
- Test performance with large graphs

Note: This is not a failing requirement but should show reasonable performance.

Yes No

Algorithm explanation

The learner can explain their algorithm complexity, design decisions, and trade-offs made in their implementation. They should be able to discuss their pathfinding approach, capacity management, and optimization strategies.

Yes No

Performance Benchmarks

Easy maps

Test performance against reference benchmarks using provided maps:
Does the algorithm solve the **Easy maps** in less than 10 turns on average?

Reference targets (for information only):

- Linear path (2 drones): ≤ 6 turns
- Simple fork (4 drones): ≤ 8 turns
- Basic capacity (4 drones): ≤ 6 turns These reference values are provided as optimization goa learners evaluate and tune their algorithms.

Yes No

Medium maps

Test performance against reference benchmarks using provided maps:
Does the algorithm solve the **Medium maps** in 10–30 turns on average?

Reference targets (for information only):
- Dead end trap (5 drones): ≤ 12 turns
- Circular loop (6 drones): ≤ 15 turns
- Priority puzzle (5 drones): ≤ 12 turns
These reference values are provided as optimization goals to help learners evaluate and tune their algorithms.

Yes No

Hard maps

Test performance against reference benchmarks using provided maps:
Does the algorithm solve the **Hard maps** in less than 60 turns on average?

Reference targets (for information only):

- Maze nightmare (8 drones): ≤ 30 turns
- Capacity hell (12 drones): ≤ 35 turns
- Ultimate challenge (15 drones): ≤ 45 turns These reference values are provided as optimizat learners evaluate and tune their algorithms.

Yes No

Edge Cases and Error Handling

Edge cases

Test edge cases:

- Single drone scenarios
- Maps with bottlenecks or limited capacity
- Disconnected graphs (should handle gracefully)
- Invalid connections between zones
- Zero or very high capacity values

At least 4 out of 5 edge cases should be handled correctly.

Yes No

Error handling

The program provides appropriate error handling for invalid inputs:

- Malformed files with new format requirements
- Missing start_hub or end_hub zones
- Invalid capacity values
- Disconnected graphs
- Invalid zone types or connections

Error messages should be clear and informative.

Yes No

Code Quality and Documentation

Code quality

Code quality assessment:

- Code is well-structured and readable
- Proper use of object-oriented principles
- Appropriate comments and documentation
- Consistent coding style
- Visual representation code is well-integrated

Note: This is not a failing requirement but contributes to overall quality.

Yes No

Quick Live Coding modifications

Live coding

Please ask the reviewee to add a simple modification to their drone simulation program. Ask them to add a "--capacity-info" flag that displays capacity information during simulation, such as "Zone X: Y/Z drones, Connection A-B: Y/Z capacity used" for each turn.

For example, "./main.py --capacity-info map.txt" should output the normal simulation plus capacity usage information. The reviewee should be able to locate the relevant parsing and output code, make the necessary modifications, and demonstrate that it works with a test case. The entire task, including the demonstration, should take no more than 10 minutes. Was this procedure followed and did everything work correctly?

Yes No

Bonus Features

Exceptional Performance

The Performance 'perfectly' meets the performance reference targets for all provided maps. 'Perfectly' meaning the algorithm meets exactly or outperforms the number of turns specified in the reference targets for each map category.
Targets are:
Easy maps:
- Linear path (2 drones): ≤ 6 turns
- Simple fork (4 drones): ≤ 8 turns
- Basic capacity (4 drones): ≤ 6 turns
Medium maps:
- Dead end trap (5 drones): ≤ 12 turns
- Circular loop (6 drones): ≤ 15 turns
- Priority puzzle (5 drones): ≤ 12 turns
Hard maps:
- Maze nightmare (8 drones): ≤ 30 turns
- Capacity hell (12 drones): ≤ 35 turns
- Ultimate challenge (15 drones): ≤ 45 turns

Yes No

Challenger Map

The Impossible Dream map is solved and beats the reference record of 45 turns.

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

Ok Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Incomplete group Concerning situation Leaks Forbidd

Can’t support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation